

OLD DOMINION UNIVERSITY

CYSE 301 - CYBERSECURITY TECHNIQUES AND OPERATIONS

Assignment #3 - Sword vs. Shield

James Davis

UIN: 01270342

INSTRUCTIONS

In this assignment, you will act as an attacker to identify the vulnerabilities in the LAN network and a defender to apply proper countermeasures. You need to provide a screenshot for each task below.

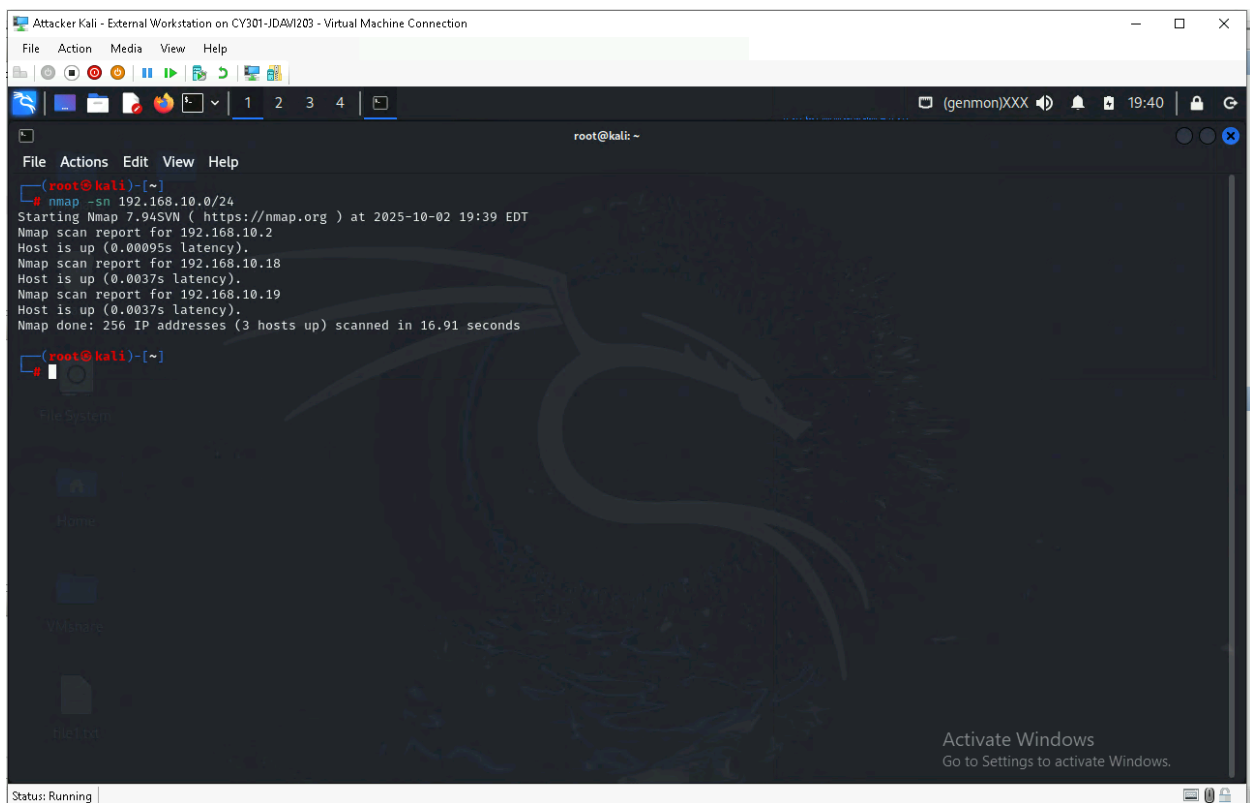
TASK A: SWORD - NETWORK SCANNING (20 + 20 = 40 POINTS)

Power on the listed VMs and complete the following steps from the **External Kali** (you can use either nmap or zenmap to complete the assignment)

- External Kali
- pfSense
- Ubuntu
- Windows Server 2022

Make sure you didn't add/delete any firewall policy before continuing.

1. Use Nmap to profile the basic information about the **subnet** topology (including open ports information, operation systems, etc.). You need to get the **service** and **backend software** information associated with each opening port in each VM.



```
Attacker Kali - External Workstation on CY301-JDA/1203 - Virtual Machine Connection
File Action Media View Help
1 2 3 4
(genmon)XXX 19:40
root@kali: ~
File Actions Edit View Help
root@kali: ~
nmap -sn 192.168.10.0/24
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-10-02 19:39 EDT
Nmap scan report for 192.168.10.2
Host is up (0.00095s latency).
Nmap scan report for 192.168.10.18
Host is up (0.0037s latency).
Nmap scan report for 192.168.10.19
Host is up (0.0037s latency).
Nmap done: 256 IP addresses (3 hosts up) scanned in 16.91 seconds
root@kali: ~
Add System
Hacker
Virtual
Activate Windows
Go to Settings to activate Windows.
Status: Running
```

Figure 1 – The output of the “nmap -sn 192.168.10.0/24” command. This uses the necessary parameters to determine which hosts in the subnet are currently online.

```

Attacker Kali - External Workstation on CY301-JDAW1203 - Virtual Machine Connection
File Action Media View Help
1 2 3 4
root@kali: ~
File Actions Edit View Help
Nmap scan report for 192.168.10.18
Host is up (0.0037s latency).
Nmap scan report for 192.168.10.19
Host is up (0.0037s latency).
Nmap done: 256 IP addresses (3 hosts up) scanned in 16.91 seconds

root@kali: ~
# nmap -sV -p- 192.168.10.18 192.168.10.19
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-10-02 19:41 EDT

root@kali: ~
# nmap -sV -p 1-1000 192.168.10.18 192.168.10.19
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-10-02 19:56 EDT
Nmap scan report for 192.168.10.18
Host is up (0.0086s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
20/tcp    closed ftp-data
21/tcp    open  ftp    vsftpd 3.0.5
22/tcp    open  ssh    OpenSSH 8.9p1 Ubuntu 3ubuntu0.7 (Ubuntu Linux; protocol 2.0)
990/tcp    closed ftps
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Nmap scan report for 192.168.10.19
Host is up (0.0066s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
135/tcp   open  msrpc   Microsoft Windows RPC
139/tcp   open  netbios-ssn Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds?
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 2 IP addresses (2 hosts up) scanned in 26.75 seconds

root@kali: ~
#
Status: Running

```

Figure 2 – The output of the “`nmap -sV -p 1-1000 192.168.10.18 192.168.10.19`” command. Please note that while I attempted to run the same command with the “`-p`” parameter (to scan all 65,535 ports), I ultimately had to cancel it because it did not seem to complete anytime soon. It seemed more practical to scan the top 1,000 ports in an academic lab environment. The “`-sV`” parameter will detect the service name and version of each open port.

```

Attacker Kali - External Workstation on CY301-JDAW203 - Virtual Machine Connection
File Action Media View Help

root@kali: ~
File Actions Edit View Help

(root@kali)~# sudo nmap -O 192.168.10.18 192.168.10.19
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-10-02 20:00 EDT
Nmap scan report for 192.168.10.18
Host is up (0.017s latency).
Not shown: 968 filtered tcp ports (no-response), 30 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
Device type: general purpose|storage-misc|firewall|webcam
Running (JUST GUESSING): Linux 2.6.X|4.X|3.X|5.X (92%), Synology DiskStation Manager 5.X (86%), WatchGuard Firewall 11.X (86%), Tandberg embedded (85%)
OS CPE: cpe:/o:linux:linux_kernel:2.6.32 cpe:/o:linux:linux_kernel:4.4 cpe:/o:linux:linux_kernel:3.10 cpe:/o:linux:linux_kernel:5 cpe:/o:linux:linux_kernel
cpe:/a:synology:diskstation_manager:5.1 cpe:/o:watchguard:fireware:11.8 cpe:/h:tandberg:vcs
Aggressive OS guesses: Linux 2.6.32 (92%), Linux 4.4 (92%), Linux 2.6.32 or 3.10 (91%), Linux 2.6.32 - 2.6.35 (90%), Linux 2.6.32 - 2.6.39 (90%), Linux 4.0
(89%), Linux 5.0 - 5.4 (88%), Linux 3.11 - 4.1 (88%), Linux 3.2 - 3.8 (88%), Linux 2.6.18 (87%)
No exact OS matches for host (test conditions non-ideal).

Nmap scan report for 192.168.10.19
Host is up (0.015s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: general purpose
Running (JUST GUESSING): Microsoft Windows 2022|11|2016|10 (95%)
OS CPE: cpe:/o:microsoft:windows_server_2016 cpe:/o:microsoft:windows_10
Aggressive OS guesses: Microsoft Windows Server 2022 (95%), Microsoft Windows 11 21H2 (91%), Microsoft Windows Server 2016 (91%), Microsoft Windows 10 (86%)
No exact OS matches for host (test conditions non-ideal).

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 2 IP addresses (2 hosts up) scanned in 23.50 seconds

(root@kali)~#

```

Figure 3 – The output of the “`sudo nmap -O 192.168.10.18 192.168.10.19`” command, which is used to detect the OS running on the machines that are using the provided IP addresses.

Subnet topology information, according to Nmap

Host IP address	Open TCP ports	Services & versions	OS guesses
192.168.10.2 (pfSense Firewall LAN interface)	N/A	N/A	N/A
192.168.10.18 (Ubuntu 22.04 VM)	21 22	21/tcp ftp – vsftpd 3.0.5 22/tcp ssh – OpenSSH 8.9p1 “Ubuntu 3ubuntu0.7”	Linux kernel 2.6–5.x (no exact match)
192.168.10.19 (Windows Server 2022 VM)	135 139 445	135/tcp msrpc – Microsoft Windows RPC 139/tcp netbios-ssn 445/tcp microsoft-ds	Windows Server 2022 (highest match), OS scan “conditions non-ideal”

Explanation:

In Task A.1, I started by running an Nmap command to determine which hosts in the subnet are currently online. This resulted in 3 IP addresses being shown, which represented the LAN interface of the pfSense gateway/firewall software (since the pfSense software functions on both LAN and WAN interfaces), the Ubuntu 22.04 VM, and the Windows Server 2022 VM. I continued Task A.1 to run another version of the Nmap command that would check for service names and their version numbers on each open port. Lastly, I finished Task A.1 by running yet another version of the Nmap command that ran an OS detection on the specified IP addresses (which were for the Ubuntu VM and Windows Server VM).

2. Run Wireshark in Internal Kali VM while External Kali is scanning the network. Discuss the traffic pattern you observed. What do you find? **Please write a 200-word essay to discuss your findings.**

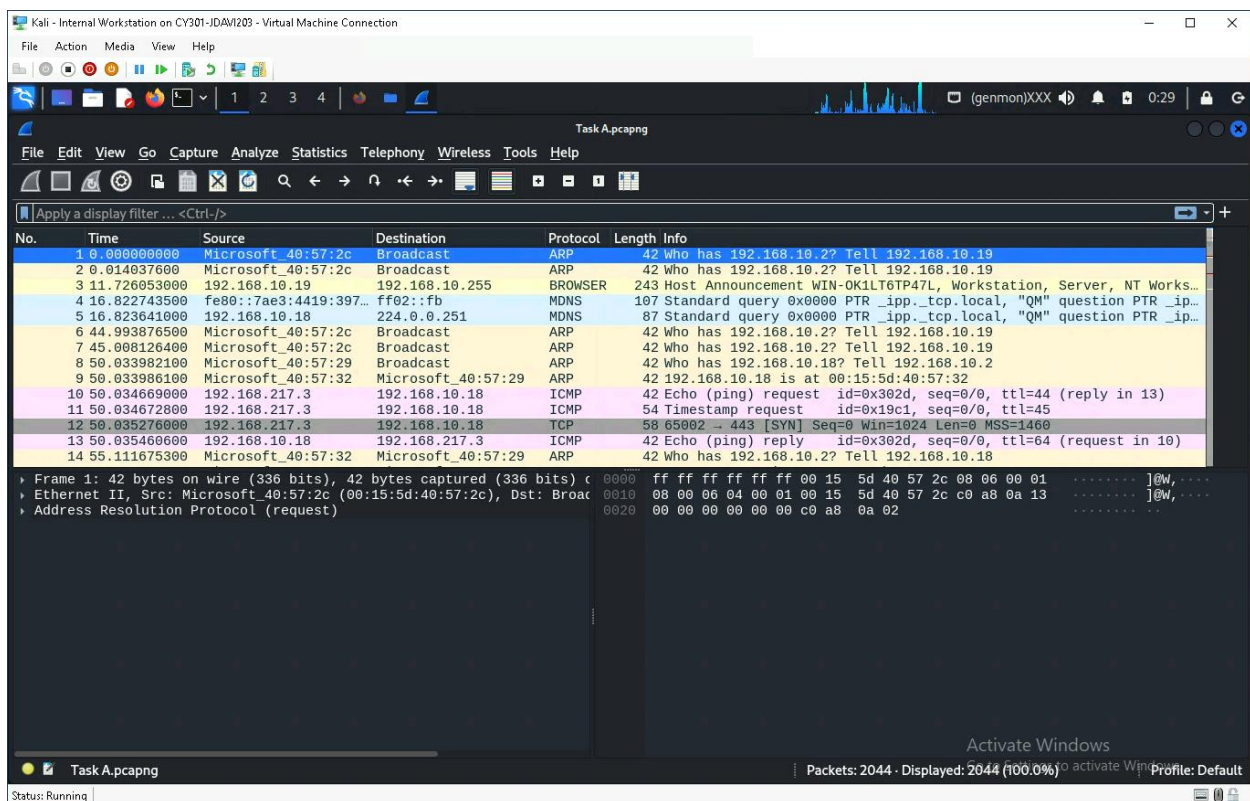


Figure 4 – A sample screenshot of the Wireshark packet capture file I saved and used for Task A.2.

While I was capturing packets in Wireshark on the Internal Kali VM as Nmap was running a scan from the External Kali VM, the capture file suggested that the External Kali VM

(192.168.217.3) was gathering a lot of information from the Ubuntu VM (192.168.10.18). This was mainly noticeable in the form of a substantial amount of TCP SYN packets that were being sent from the External Kali VM, which were checking to see which ports were open or closed. For example, if a port was open, a SYN-ACK reply was observed. If a port was closed, then an RST reply was observed. Otherwise, there would be no reply, if the SYN packet was dropped. There were also 2 packets found using “tcp.port == 53” as the display filter, meaning that DNS queries were heard, which suggests that Nmap tried to do reverse lookups. Lastly, ICMP echo requests & replies were both observed, which would’ve occurred during the host discovery phase.

Interestingly, despite executing the “nmap -sV -p 1-1000 192.168.10.18 192.168.10.19” command from the External Kali VM, I only observed 1 packet using the “ip.addr == 192.168.10.19” display filter, which was Windows Server producing a broadcast packet using the BROWSER protocol; this appears to be background network traffic that is unrelated to this lab activity.

TASK B: SHIELD – PROTECT YOUR NETWORK WITH A FIREWALL (10 + 10+ 20 + 20 = 60 POINTS)

In order to receive full credits, you need to fill the table (add more rows if needed), implement the firewall rule(s), show me the screenshot of your firewall table, and verify the results.

1. Configure the pfSense firewall rule to block the ICMP traffic from External Kali to Ubuntu VM.

Rule #	Interface	Action	Source IP	Destination IP	Protocol (port # if applicable)
1	WAN	Block	192.168.217.3	192.168.10.18	ICMP

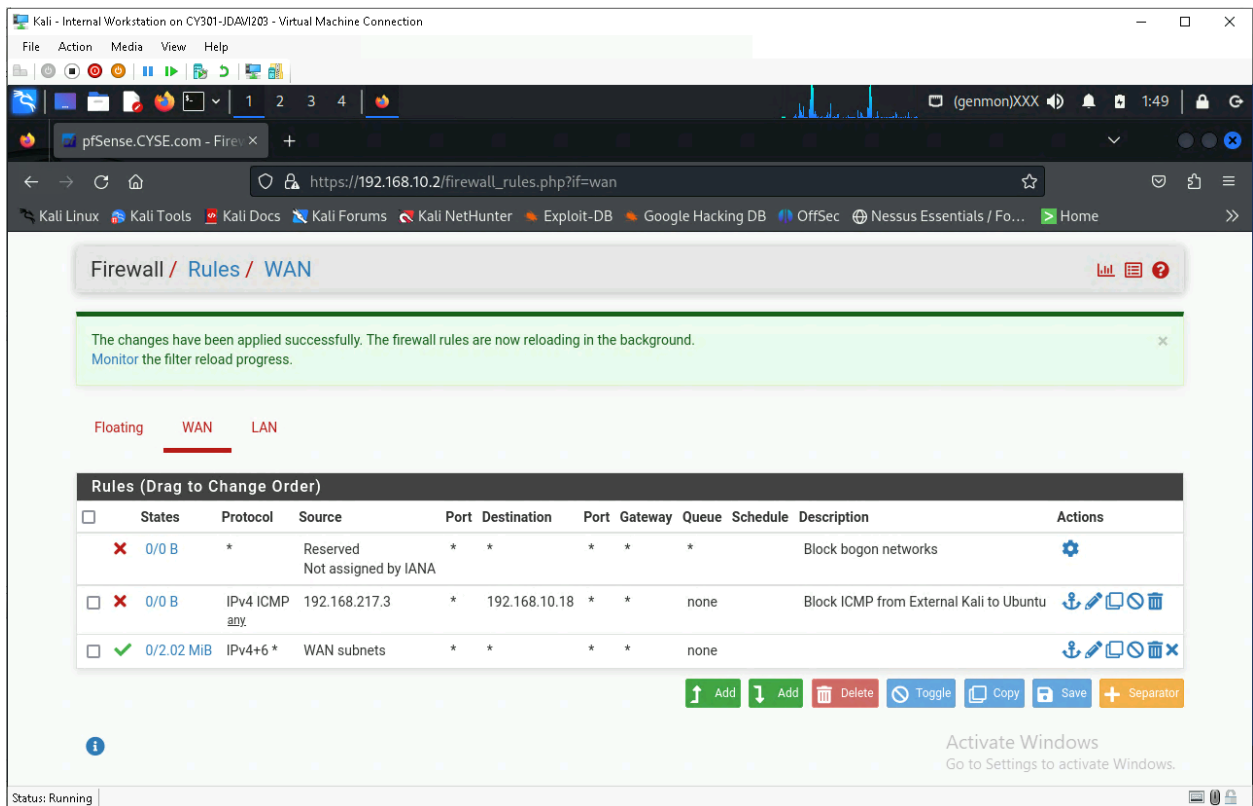


Figure 5 – A screenshot of all firewall rules that I had on the WAN interface of the pfSense software. The first and last rules were already included by default; I added the 2nd rule (located in the middle of this screenshot).

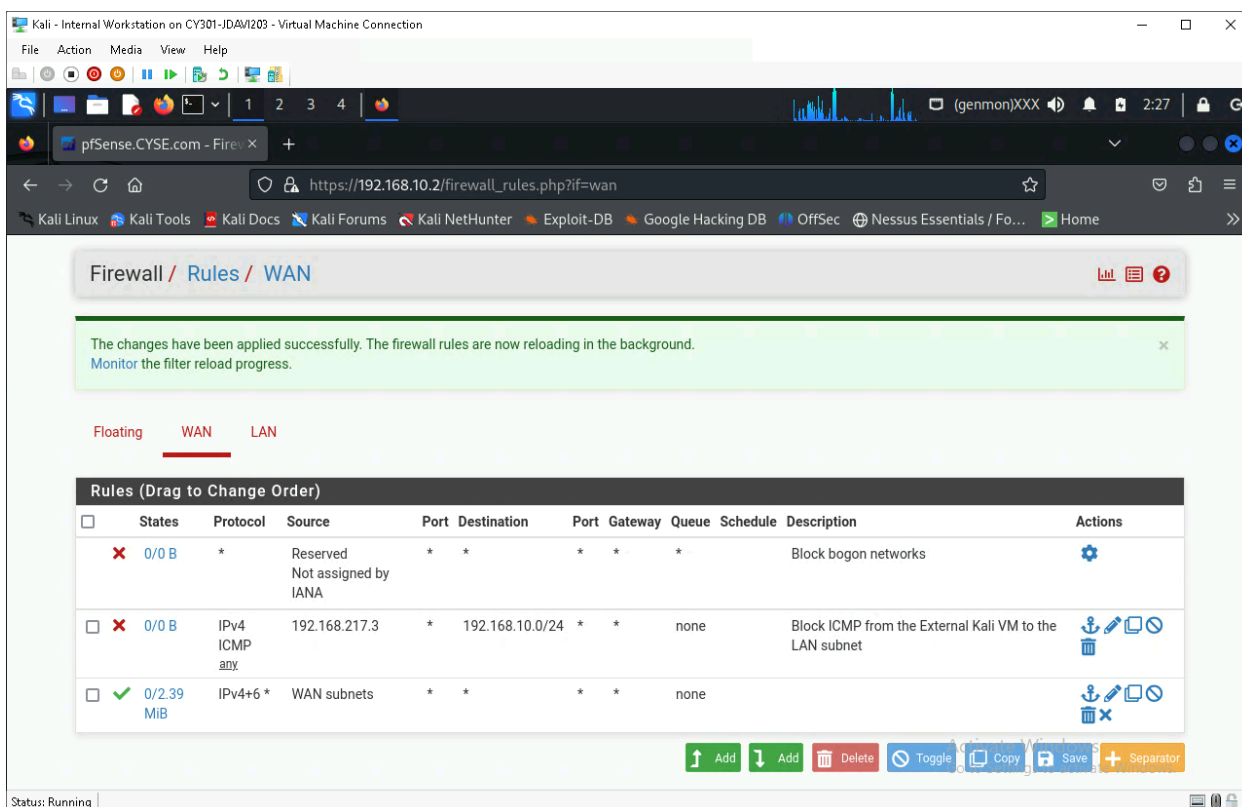


Figure 7 – Another screenshot of all firewall rules that I had on the WAN interface of the pfSense software. The first and last rules were already included by default; I added the 2nd rule (located in the middle of this screenshot).

Explanation:

Clear the previous firewall policies and configure the pfSense firewall to block ALL traffic from External Kali to the LAN side, except for the FTP protocol towards Ubuntu.

Rule #	Interface	Action	Source IP	Destination IP	Protocol (port # if appliable)
1	WAN	Pass	192.168.217.3	192.168.10.18	TCP (21)
2	WAN	Block	192.168.217.3	192.168.10.0/24	Any

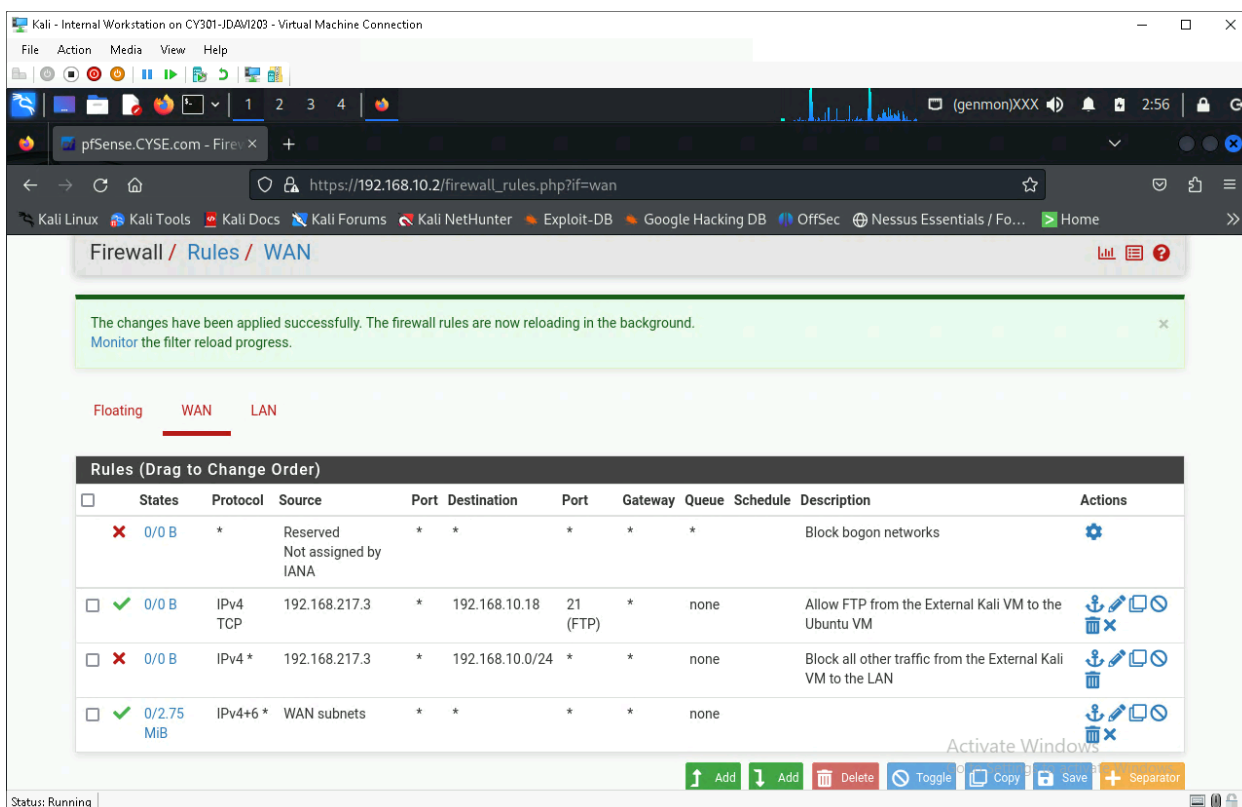


Figure 9 – Another screenshot of all firewall rules that I had on the WAN interface of the pfSense software. The first and last rules were already included by default; I added the 2nd and 3rd rules (located in the middle of this screenshot).

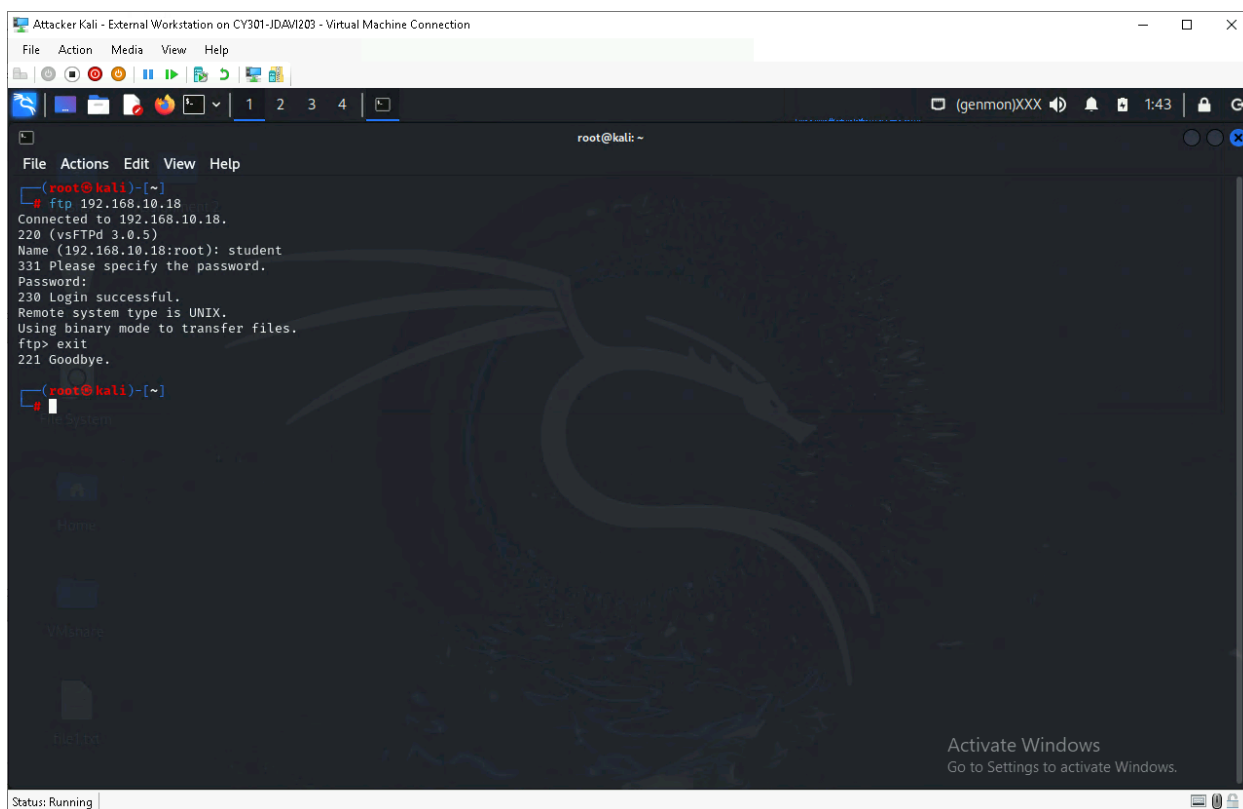


Figure 10 – A screenshot of the External Kali VM successfully attempting to FTP into the Ubuntu VM.

```

Attacker Kali - External Workstation on CY301-JDAW1203 - Virtual Machine Connection
File Action Media View Help

root@kali: ~
File Actions Edit View Help

(root@kali)~# ping -c 4 192.168.10.18
PING 192.168.10.18 (192.168.10.18) 56(84) bytes of data.
--- 192.168.10.18 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3072ms

(root@kali)~# ping -c 4 192.168.10.19
PING 192.168.10.19 (192.168.10.19) 56(84) bytes of data.
--- 192.168.10.19 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3068ms

(root@kali)~# nmap -p 22,80,443 192.168.10.18
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-10-03 03:05 EDT
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.09 seconds

(root@kali)~# nmap -Pn -p 22,80,443 192.168.10.18
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-10-03 03:10 EDT
Nmap scan report for 192.168.10.18
Host is up.

PORT      STATE SERVICE
22/tcp    filtered ssh
80/tcp    filtered http
443/tcp   filtered https

Nmap done: 1 IP address (1 host up) scanned in 16.11 seconds

(root@kali)~#

```

Activate Windows
Go to Settings to activate Windows.

Status: Running

Figure 11 – A screenshot providing evidence of how ICMP transmissions and other services not working, despite FTP working from External Kali to Ubuntu, due to the new combination of firewall rules used in Task B.3.

Explanation:

Because the ‘FTP allow’ rule is positioned above the ‘block all traffic to the network’ rule, FTP still works from External Kali to Ubuntu, while External Kali is simultaneously unable to ping the same computer (192.168.1.18; the Ubuntu VM), nor any other computer on the same network (such as 192.168.1.19; the Windows Server VM). To test other services from External Kali to Ubuntu, I used a couple of Nmap commands to scan some commonly used ports. Since all of them came back as “filtered,” we know that traffic is blocked for these, unlike FTP.

4. Keep the firewall policies you created in Task B.3 and repeat Task A.1. What’s the difference?

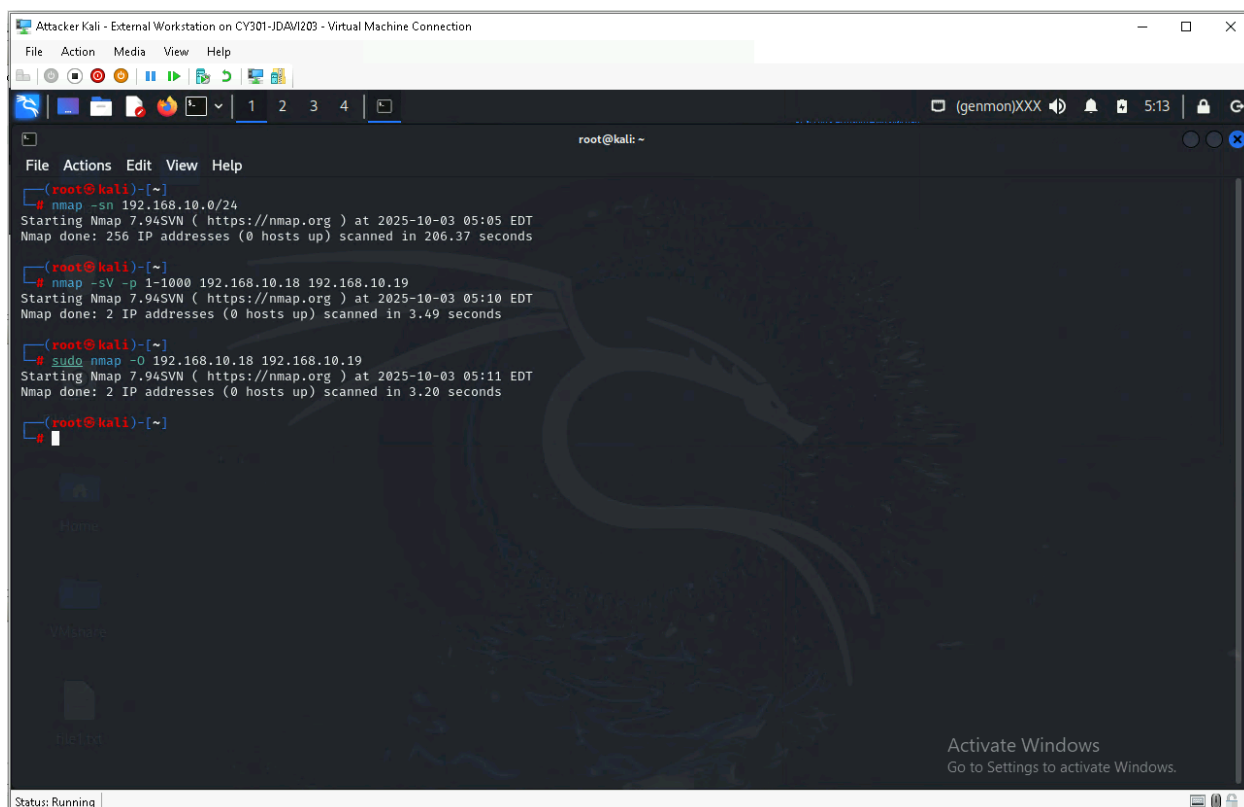


Figure 12 – A screenshot confirming that the External Kali VM is no longer able to use Nmap to find online hosts, look for open ports, or detect operating systems on hosts located in the LAN.

In Task B.3, I created two firewall policies that collectively block all traffic from the External Kali VM to the LAN (except for the FTP service to the Ubuntu VM). By repeating the Task A.1 scans (consisting of host discovery, service scan, and OS detection), it is no longer possible for the External Kali VM to gather the considerable amount of information that it was originally able to without additional firewall rules.

For example, the ping sweep (`nmap -sn 192.168.10.0/24`) was originally able to see 3 hosts online, including the pfSense firewall, the Ubuntu 22.04 VM, and the Windows Server 2022 VM. However, Nmap now behaves as if the entire subnet (consisting of 256 IP addresses) is offline; this is due to ICMP echo requests being blocked.

Additionally, the service/version scan (`nmap -sV -p 1-1000 192.168.10.18 192.168.10.19`) and the OS detection (`sudo nmap -O 192.168.10.18 192.168.10.19`) both fail their attempts to gather specific types of information. While both of these commands were still able to complete more efficiently compared to the ping sweep (2 IP addresses to be scanned vs. 256 IP addresses to be scanned), the firewall blocks everything except FTP to the Ubuntu VM, which is why open ports can no longer be detected, and OS fingerprinting attempts can no longer be made.

EXTRA CREDIT (15 POINTS)

Use NISSUS to enumerate the security vulnerabilities of Microsoft Windows Server 2022 VM in the CCIA network.

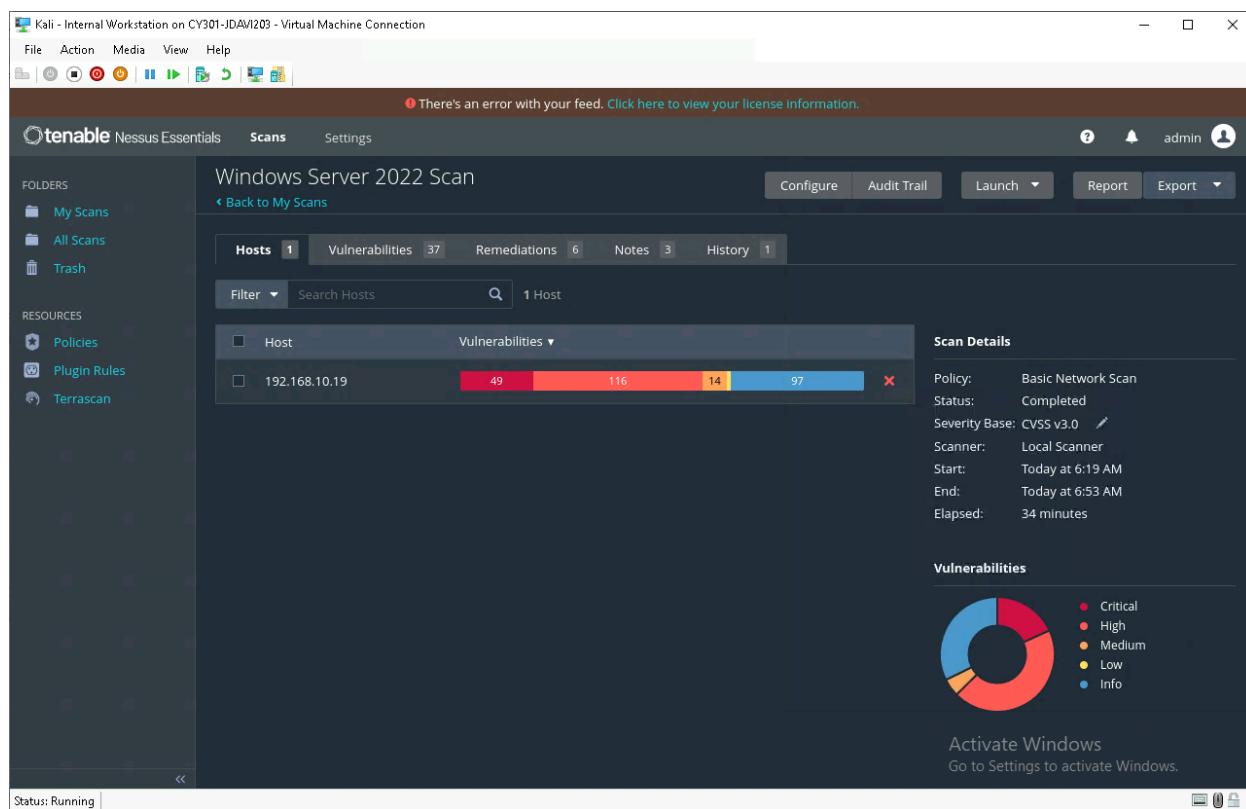


Figure 13 – The initial perspective shown of a scan following the completion of a basic network scan. This point of view includes multiple graphical displays consisting of each level of risk across all vulnerabilities detected.

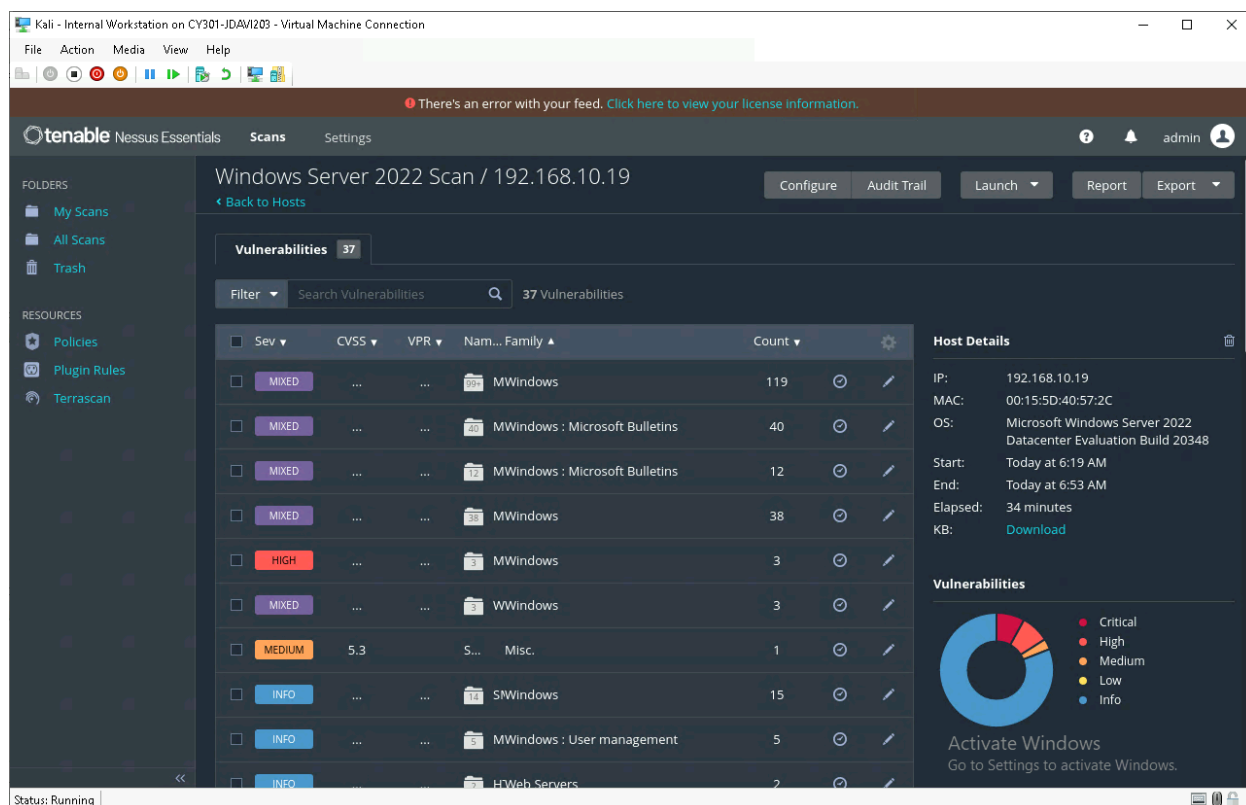


Figure 14 – A screenshot of the top of the list shown after clicking on the “Vulnerabilities” tab. Note the “Count” column, which shows the categories that likely deserve the most attention.

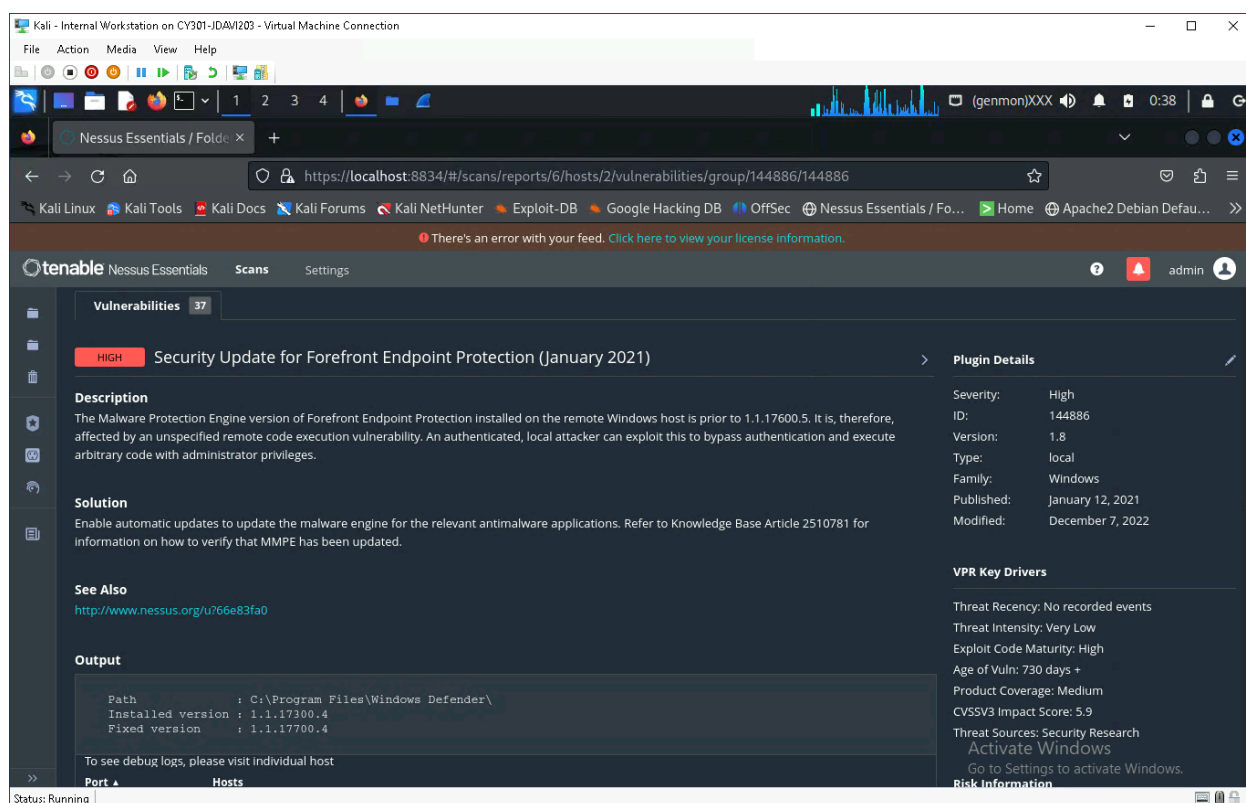


Figure 15 – The top of a sample vulnerability that I chose to analyze. This allows a system administrator to view a description, a solution, and the severity of the vulnerability.

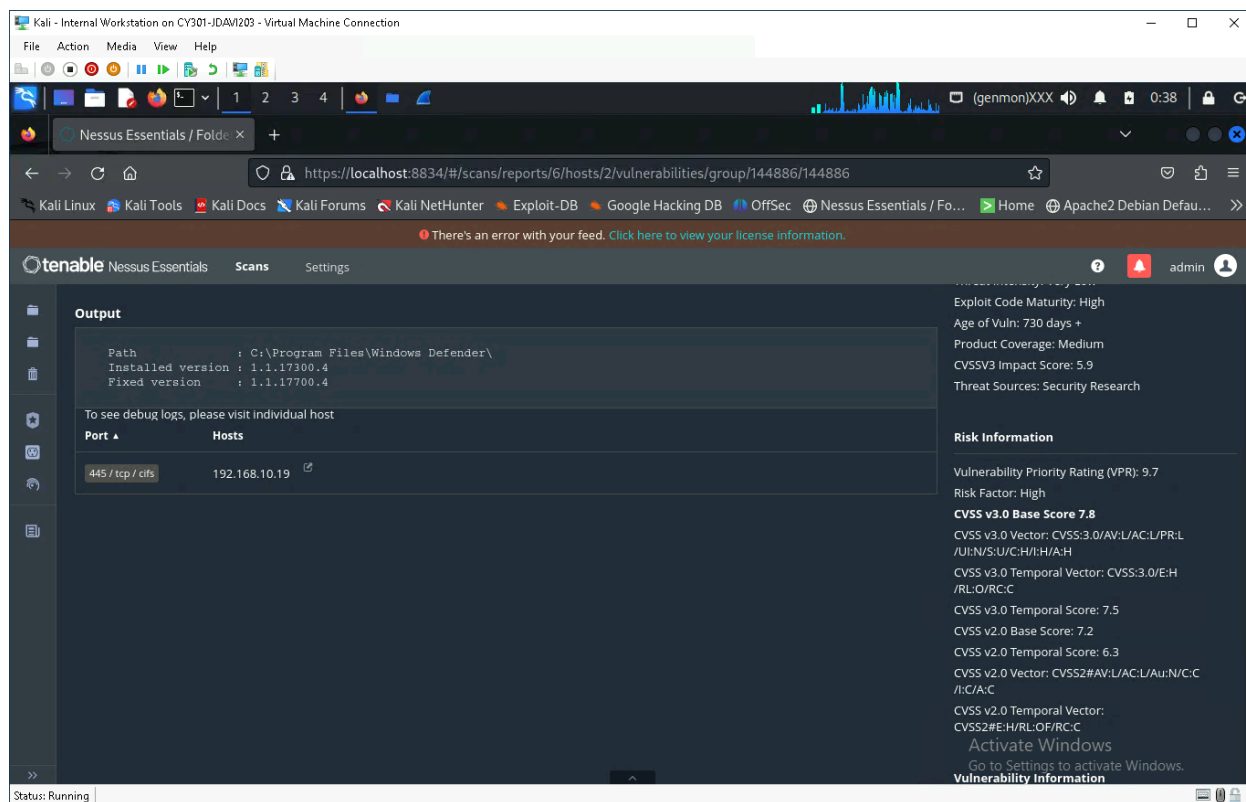


Figure 16 – A continuation of the same sample vulnerability as previously shown in Figure 15. This shows the VPR score, and numerous CVSS scores.

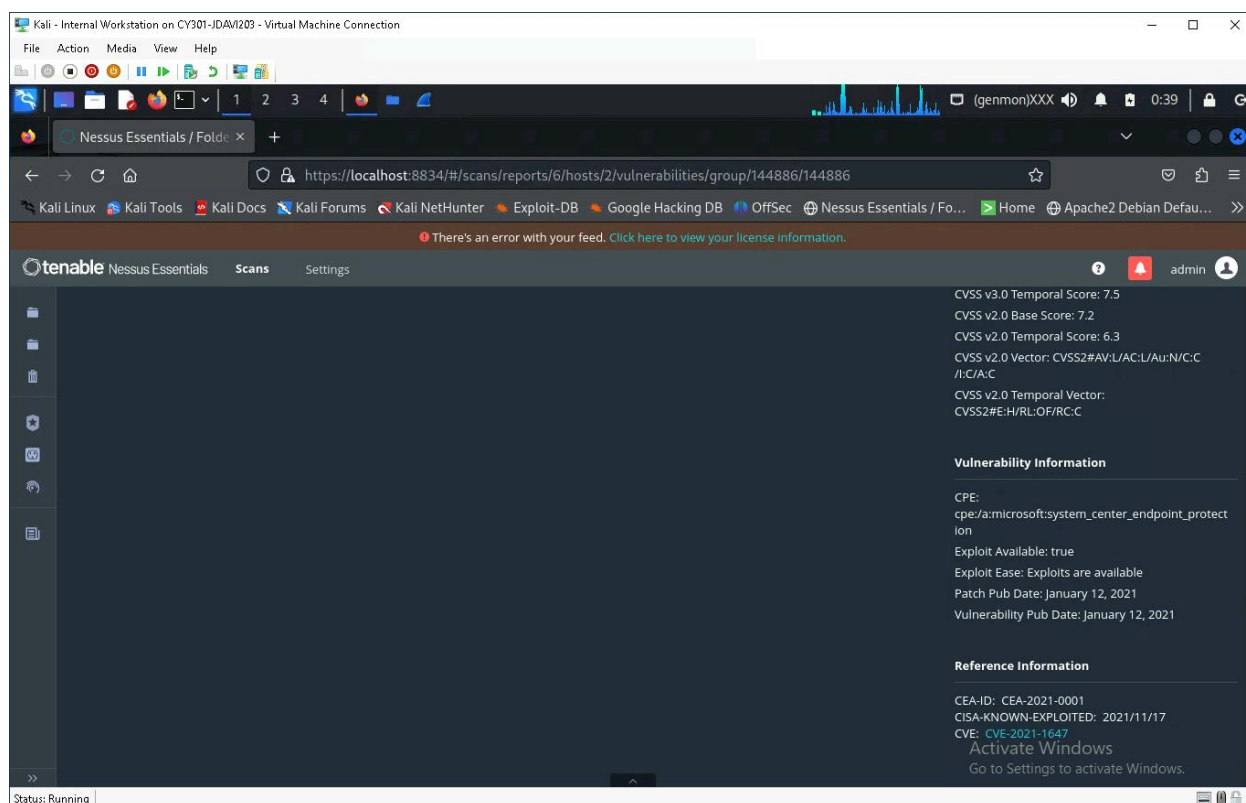


Figure 17 – The end of the sample vulnerability I analyzed in this lab activity. Note the “Reference Information,” which includes a CVE link to the NIST website for even more information about the vulnerability’s age, the software configurations it’s known to affect, and resources to patch the vulnerability.

I completed this lab activity by running Nessus (at <https://localhost:8834>) on the Internal Kali VM, targeting the local IP address of the Windows Server 2022 VM (at 192.168.10.19), and inputting the Windows login credentials. The “Basic Network Scan” lasted for a total of 34 minutes, and resulted in a total of 49 critical severity vulnerabilities, 116 high severity vulnerabilities, 14 medium severity vulnerabilities, 1 low severity vulnerability, and 97 additional pieces of information (shown in the blue sector of the results).

By clicking on the “Vulnerabilities” tab, a list of categories of vulnerabilities can be listed from the highest to lowest count per category, with Microsoft Edge having the most, at 119 total vulnerabilities to be addressed. Other categories of interest include Microsoft .NET Framework (12 vulnerabilities reported), Microsoft System Center Endpoint Protection (3 vulnerabilities reported), the SMB (Server Message Block) network protocol (15 vulnerabilities reported), and HTTP (Hypertext Transfer Protocol, for web services) (2 vulnerabilities reported).

If I view additional details about one of the numerous unpatched vulnerabilities (such as a high severity entry labelled “Security Update for Forefront Endpoint Protection (January 2021)”), a wealth of information is provided about the vulnerability, including a description of it, a solution for it, a VPR (Vulnerability Priority Rating) score, numerous types of CVSS scores, and

reference information (including a CVE link). As a result, it's clear how Nessus is a prime example of a vulnerability scanning software that can help system administrators patch their systems in a proactive manner.